实验二：C-- 语义分析

闵振昌 241880334

1. 实验功能与实现方法

本实验在实验一的 Flex/Bison 词法与语法分析器上增加语义分析，实现必做的 1-17 类错误检查，默认只启用选做 2.2（嵌套作用域），对应平台第 2 组选做范围。程序从源文件构造语法树后再进行分析；合法输入不输出内容，错误通过标准输出报告编号、行号和说明。

1.1 模块与类型表示

syntax.y 负责文法与语法树，lexical.l 负责 token；新增 semantic.c、semantic.h 和共享结点头文件 tree.h。语义模块按源文件顺序遍历定义、语句和表达式，默认保持“先定义，再使用”的规则，进入函数体前登记函数以支持递归。

类型分为 int、float、数组和结构体。数组递归保存元素类型与每维大小；比较数组时忽略大小，只比较维数和基类型。结构体保存有序域链表，成员访问按域名查找，域名只在所属结构体内部查重。

1.2 符号表与表达式检查

变量和结构体标签使用散列表，并通过作用域链记录本层符号。新符号插入桶首；退出语句块时按逆序移除本层符号，恢复外层同名定义。函数参数与函数体最外层局部变量共享作用域，因此同层重定义仍报错，内层遮蔽合法。函数使用独立表记录返回类型、参数类型和定义状态，在进入函数体前登记以支持直接递归。

每个表达式返回类型和左值属性。赋值检查左值与两侧类型；算术与关系运算要求相同的基本类型，逻辑运算和条件要求 int；函数调用逐项比较实参与形参。数组访问检查基对象与下标，结构体访问检查基对象与域。NULL 类型表示已诊断的错误，抑制由未定义名字引起的次生类型错误，并继续检查其他独立错误。

1.3 选做规则与兼容模式

提交默认仅启用选做 2.2：嵌套块和不同函数的局部变量可以重名，同层重复定义仍报错。结构体保持名等价，同一定义共享类型标识，不同名或不同匿名定义不等价。函数声明属于当前模式不支持的语法，在报告 Type B 后终止语义检查，避免次生错误。--basic 用于对照原始必做规则；--all 只用于本地验证其他扩展，不是提交默认模式。

2. 编译与运行方法

需要 Linux/WSL 环境中的 GCC、Flex、Bison 和 libfl；语义实现只依赖 C 标准库，不使用额外第三方 C 库。进入提交包的 Code 目录执行：

```
cd Code
make clean
make
./parser path/to/input.cmm
./parser --basic path/to/input.cmm
```

Makefile 先生成 syntax.tab.c/.h，再生成 lex.yy.c，将扫描器包含在语法分析器中；syntax.tab.o 与 semantic.o 分别编译后链接得到 parser。一个输入文件对应一次运行；也可从标准输入读取。处理语义错误后正常返回 0，文件打开失败或命令行参数错误返回 1。

错误格式示例：

```
Error type 5 at Line 4: Type mismatched for assignment.
```

3. 测试与结果

测试脚本 tests/run_tests.py 共包含 148 项测试，精确比较错误编号与行号，并要求合法输入完全无输出。默认模式按选做 2.2 的规则核对手册的 17 个必做样例和 6 个选做样例，并专门检查未启用的函数声明与结构等价，另外包含必做模式和本地完整扩展模式的对照测试。

重点用例包括：多维数组忽略大小但区分维数；结构体域顺序与嵌套数组不能展开比较；同层参数重定义与内层遮蔽；退出块后变量消亡；直接递归、禁用函数声明时不产生次生错误，以及本地完整扩展模式的声明与定义冲突；实参数量和类型错误；错误类型传播及多个独立错误。

实测环境为 WSL Ubuntu 24.04，GCC 13.3.0、Flex 2.6.4、Bison 3.8.2。严格编译选项 -Wall -Wextra -Werror -pedantic 下编译通过，148/148 项测试通过。AddressSanitizer/UndefinedBehaviorSanitizer 检测版本同样全部通过，未发现内存越界、泄漏或未定义行为。沿用 lab1 的参考 Makefile 验证时，本机缺少 liby，去除冗余的 -ly 链接项后，编译并通过全部测试。

```
python3 tests/run_tests.py Code/parser
```

验收脚本另存于 lab2/tests，不放入提交 ZIP；仅验收脚本需要 Python 3，parser 不依赖 Python。

语法树结点及语义分析期间分配的类型、域、符号和作用域统一回收。提交包仅包含 Code/ 源文件夹与 report.pdf，不包含生成的扫描器、分析器 C 文件、目标文件或平台相关的可执行文件。